FitTrail

Personal Workout & Diet Tracker

Official Project Architecture & Technical Documentation

Project	Flutter Application Project Examination (Problem Statement 6)
Prepared by	Sarthak Kanade
Document version	1.0
Date	3 October 2026
Platform targets	Android, iOS, Web, macOS, Windows, Linux
Design file	Figma: FitTrail - UI Design (figma.com/design/LEl1qwbUNcWHg4H0y9oXw5)

Contents

Contents 2
1. Introduction 3
2. Requirements Traceability 3
3. Features 4
4. System Architecture 4
5. State Management and Data Flow 5
6. Navigation and Screens 6
7. Custom Widgets 7
8. Business Logic 7
9. Local Storage 8
10. UI and Design 8
11. Design Justification 10
12. Testing and Quality 10
13. Build and Deployment 11
14. Limitations and Future Scope 11
Appendix A. Project Files 12

1. Introduction

1.1 Purpose

This document describes the design, architecture and implementation of FitTrail, a personal workout and diet tracker built with Flutter. It is written to support the project examination: it maps each requirement of the problem statement to a concrete feature, explains the technical choices that were made, and records the limits of the current version.

1.2 Problem statement

FitTrail lets a user log daily workouts and meals, then asynchronously computes the calorie balance for the day so the user always knows whether they are within their target. The solution must include proper justification for the choices made.

1.3 Scope

FitTrail is an offline-first, single-user application. All data is stored on the device. There is no server, no account system and no network dependency for core functionality. The application provides three main screens (Daily Summary, Log Meal, Log Workout), edit and delete of entries, a configurable daily target, and a responsive layout for phones, tablets and desktop.

1.4 Summary of the solution

The app is organised in layers: presentation, Riverpod state, data access and business logic, and local storage. Logging a meal or workout writes to local storage asynchronously, updates a shared Riverpod state, and triggers an asynchronous recalculation of the calorie balance. The same balance state is read by the Summary screen and by both logging screens, so all three always agree.

2. Requirements Traceability

The table maps every objective, outcome and deliverable in the problem statement to the part of the system that meets it.

ID	Requirement (from problem statement)	How it is met
R1	Log Workout, Log Meal and Daily Summary screens using Form, ListView and Card widgets	Three screens; Form + validators for entry; ListView of Card-based entries (Section 6)
R2	Material 3 theme with progress rings and clear calorie typography	Material 3 theme; CalorieRing custom widget; large displayMedium remaining-calories figure (Sections 7, 10)
R3	async/await for the calorie-balance calculation	dayLogProvider (AsyncNotifier) and balanceProvider (FutureProvider) await storage results before calculating (Section 5)
R4	Riverpod exposes the result to the Summary screen	balanceProvider is watched by SummaryScreen, LogMealScreen and LogWorkoutScreen
R5	Figma flow with progress cues	8 Figma frames, step indicator on forms, progress ring on Summary (Section 10)
R6	Each workout entry shows exercise type, duration and estimated calories burned	Workout EntryCard shows activity, duration and MET-based kcal (Sections 6, 8)
R7	Logging triggers async recalculation	addEntry -> repository write -> state update -> balanceProvider recomputes (Section 5)
R8	Progress ring for calories consumed versus target	Outer arc of CalorieRing = consumed / target

ID	Requirement (from problem statement)	How it is met
R9	Edit or delete an entry; balance updates immediately	Tap to edit (pre-filled form); swipe to delete with Undo; both go through dayLogProvider
R10	Local storage for offline access	shared_preferences, one JSON document per calendar day (Section 9)
R11	Responsive prototype of the log-to-summary journey	Adaptive shell: bottom bar under 600 dp, NavigationRail from 600 dp
R12	Proper justification	Section 11

3. Features

3.1 Core features

- **Log Workout** - choose an activity, enter a duration, see the estimated calories burned before saving.
- **Log Meal** - food name, calories and meal category with validation.
- **Daily Summary** - progress ring, eaten / burned / remaining figures, and entry lists grouped into Meals and Workouts.
- **Edit and delete** - tap an entry to edit; swipe left to delete with an Undo action.
- **Offline storage** - logs, target and weight persist on the device.

3.2 Additional features

- **Quick-add meal chips** - 10 common foods pre-fill name, calories and category.
- **Date navigation** - previous/next arrows, a date picker, a quick return to Today, and relative labels (Today, Yesterday, Tomorrow).
- **Target and profile dialog** - set body weight and a goal plan (Fat Loss, Maintain, Muscle Gain); the dialog suggests a daily target.
- **Live preview** - the logging screens show the current remaining budget, and the workout screen shows the projected effect before saving.
- **Dual-arc ring** - outer arc for food, inner arc for exercise; a status badge shows Deficit or Surplus.
- **Theme switching** - system, light and dark modes.
- **Duration controls** - numeric input, a 5 to 120 minute slider and quick chips (15, 30, 45, 60, 90).

4. System Architecture

4.1 Layered design

FitTrail follows a unidirectional, reactive layered architecture. Each layer only depends on the layer beneath it, so the calculation logic can be tested without any UI.

Presentation	Screens, CustomPainter widgets, dialogs and modal sheets
v ref.watch() / ref.read()	
Riverpod state	selectedDateProvider, dayLogProvider, balanceProvider, targetProvider, weightProvider, themeModeProvider
v async calls	
Data and logic	LogRepository, CalorieCalculator, MetCalculator
v JSON serialisation	

Local storage	shared_preferences (on-device key-value store)

4.2 Technology stack

Technology	Version	Purpose
Flutter SDK	>= 3.0.0 (Dart 3)	Cross-platform UI toolkit
flutter_riverpod	^2.6.1 (see note)	State management with AsyncNotifier, Notifier and FutureProvider
shared_preferences	^2.3.5	Asynchronous on-device key-value persistence
google_fonts	^6.2.1	Typography (Outfit)
intl	^0.20.2	Date formatting
uuid	^4.5.1	Unique IDs (v4) for logged entries

Version note. The project pins flutter_riverpod ^2.6.1. At the time of writing the latest release line is 3.x, in which the older StateNotifier API is moved to a legacy import. The Notifier / AsyncNotifier style used here is the style recommended going forward, so the code remains compatible in design. Check pub.dev for the current versions of all packages before upgrading.

5. State Management and Data Flow

5.1 Providers

Provider	Type	Responsibility
selectedDateProvider	Notifier<DateTime>	Active calendar day (time stripped). Methods: nextDay, previousDay, setDate, setToday.
targetProvider	AsyncNotifier<int>	Daily calorie target (default 2000 kcal). Loads and saves key fittrail_target_kcal.
weightProvider	AsyncNotifier<double>	Body weight in kg (default 70.0). Feeds the MET calculation.
dayLogProvider	AsyncNotifier<List<Entry>>	Entries for the selected date. Reloads when the date changes. Methods: addEntry, updateEntry, deleteEntry, restoreEntry (undo).
balanceProvider	FutureProvider<Balance>	Awaits dayLogProvider and targetProvider, runs CalorieCalculator, returns an immutable Balance.
themeModeProvider	Notifier<ThemeMode>	System / light / dark selection.

5.2 Asynchronous calorie-balance flow

The calculation is asynchronous because it sits directly behind storage I/O, not because of an artificial delay. The sequence for saving a meal or workout is:

Step	Component	Action
1	Form widget	User taps Save; form calls dayLogProvider.notifier.addEntry(entry)
2	dayLogProvider	await repository.addEntry(date, entry)
3	LogRepository	Serialises the entry list to JSON and writes it to shared_preferences; returns the updated list
4	dayLogProvider	Sets state = AsyncData(updatedList)

Step	Component	Action
5	balanceProvider	Detects the change; awaits dayLogProvider.future and reads targetProvider
6	CalorieCalculator	calculate(entries, target) returns Balance(consumed, burned, remaining, isOver)
7	UI	CalorieRing animates; stat tiles, status badge and live previews refresh

Edit, delete and undo use the same path (updateEntry, deleteEntry, restoreEntry), so the balance changes immediately after every modification. Because the Summary and both logging screens watch the same balanceProvider, no screen keeps its own copy of the totals.

The UI handles the three AsyncValue states: loading (a light placeholder), data (the ring and totals) and error (a message with retry).

6. Navigation and Screens

6.1 Adaptive shell

MainNavigationShell uses a LayoutBuilder to follow the available width. Below 600 dp it shows a bottom NavigationBar with three destinations (Summary, Log Meal, Log Workout). From 600 dp it shows a left NavigationRail beside an expanded content area. An IndexedStack keeps each tab alive, so scroll position and form state are preserved when switching tabs.

6.2 Daily Summary

- Date header with previous/next arrows, a calendar picker and a Today shortcut.
- Animated dual-arc CalorieRing, three stat tiles (Eaten, Burned, Remaining/Deficit) and a status banner.
- Entries grouped into Meals and Workouts, each shown as an EntryCard.
- A floating "+" button opens a bottom sheet with one-tap access to Log Meal or Log Workout.
- A tune icon opens the TargetDialog for weight and goal plan.

6.3 Log Meal

- Quick-add chips pre-fill food name, calories and category.
- Validation: name not empty; calories between 1 and 5,000.
- Meal category chosen with responsive ChoiceChips.
- Live preview of the remaining budget; Save shows a confirmation SnackBar.

6.4 Log Workout

- Activity dropdown with 7 MET activities: Walking, Running, Cycling, Gym, Yoga, Swimming, Other.
- Duration from numeric input, slider (5 to 120 minutes) or quick chips.
- Calculation card showing the estimated burn using the stored body weight.
- Projected balance banner before saving.

6.5 Edit and delete

Tapping an EntryCard opens the same form pre-filled in edit mode. Swiping an EntryCard from right to left removes it and shows a SnackBar with an Undo action that restores the entry.

7. Custom Widgets

Widget	Type	Description
CalorieRing / _DualRingPainter	StatefulWidget + CustomPainter + SingleTickerProviderStateMixin	Animated concentric arcs drawn with Canvas.drawArc. Outer arc (blue #0284C7): calories eaten versus target. Inner arc (orange #F97316): calories burned. Centre text shows remaining calories in displayMedium. Badge: Deficit (green #16A37F) or Surplus (red #EF4444). Legend: Eaten, Burned, Deficit.
EntryCard	StatelessWidget (Dismissible + Card + InkWell)	Swipe-to-delete with Undo, tap-to-edit, emoji, title, duration, time of day and a colour-coded kcal pill.
TargetDialog	ConsumerStatefulWidget (AlertDialog)	Weight input and goal plan. Suggests a target and applies it with one tap.
QuickMealChips	StatelessWidget (ListView.separated + ActionChip)	10 preset foods that pre-fill the meal form.
DateHeader	ConsumerWidget	Date arrows, date picker, relative date label.

8. Business Logic

8.1 Calorie balance

```
Remaining = Target - Eaten + Burned

Remaining >= 0  ->  within budget (shown as "Deficit")
Remaining <  0  ->  over budget   (shown as "Surplus", overAmount = |Remaining|)
```

The ring shows eaten divided by target. Burned calories are added back in the Remaining figure, so the balance is net, while the ring stays faithful to the wording of the problem statement (consumed versus target).

8.2 Estimated calories burned (MET)

```
Burned (kcal) = MET x Weight (kg) x (Duration (min) / 60)
```

Activity	MET	Burn for 70 kg, 30 min
Walking	3.5	123 kcal
Running	9.8	343 kcal
Cycling	7.5	263 kcal
Gym / Weights	5.0	175 kcal
Yoga	3.0	105 kcal
Swimming	6.0	210 kcal
Other	4.0	140 kcal

8.3 Worked example

Target 2,000 kcal. The user eats 1,420 kcal, runs for 25 minutes and walks for 20 minutes (70 kg).

Item	Calculation	Result
Running	9.8 x 70 x (25 / 60)	286 kcal
Walking	3.5 x 70 x (20 / 60)	82 kcal
Burned total	286 + 82	368 kcal
Remaining	2000 - 1420 + 368	948 kcal
Ring progress	1420 / 2000	71%

8.4 Target suggestion

```
Maintenance   = Weight (kg) x 30 kcal
Target intake = Maintenance + plan offset
```

The factor of 30 kcal per kg is a simple rule of thumb, used so the user gets a reasonable starting target. It is not a clinical estimate; the target is always editable.

9. Local Storage

Persistence uses shared_preferences with no server or cloud dependency. Logs are stored per calendar date as a JSON array; the target and weight use their own keys.

```
Key:   "fittrail_log_2026-10-03"
Value: [
  { "id": "e3b0c442-98fc-1c14-9afb-4c72fe989f61",
    "type": "meal",
    "title": "Poha & Boiled Eggs",
    "kcal": 506,
    "createdAt": "2026-10-03T08:30:00.000Z",
    "mealType": "breakfast" },
  { "id": "c1a0d883-45cd-2e24-8bba-1d94fe123e42",
    "type": "workout",
    "title": "Morning Run",
    "kcal": 343,
    "createdAt": "2026-10-03T09:15:00.000Z",
    "exerciseType": "running",
    "durationMin": 30 }
]
```

Key	Content
fittrail_log_YYYY-MM-DD	JSON array of entries for that day
fittrail_target_kcal	Daily calorie target (integer)
(weight key)	Body weight in kg (default 70.0)

Entries use a type field (meal or workout) to share one list. Loading is asynchronous, so the app opens with stored data and no network access.

10. UI and Design

10.1 Figma design

The interface was designed in Figma before implementation. The file contains 8 frames, a local colour-variable set and reusable components (EntryCard, StatTile, Chip, Button, TextField and NavBar states). Prototype links cover the log-to-summary journey: Summary to Log Meal or Log Workout, Save back to Summary, entry to Edit, and Delete to the Undo state.

#	Frame	Shows
1	Daily Summary	Ring at 71%, stat tiles, meals and workouts lists, floating action button
2	Daily Summary - over target	Ring turns red; "187 kcal over"
3	Daily Summary - empty	Empty state with call-to-action buttons
4	Log Meal	Step indicator, balance strip, quick-add chips, form, meal-type chips, today's meals
5	Log Workout	Step indicator, activity and duration, live estimate card with MET formula
6	Edit entry	Pre-filled form with Update and Delete
7	Entry deleted	Undo snackbar; balance already updated
8	Tablet Summary	NavigationRail with two-pane layout

10.2 Theme

Role	Colour	Use
Primary	#0F7B5F	Within-target ring, buttons, selected items
Primary container	#BFEBDD	Ring track, selected chips
Tertiary	#D9701A	Burned calories, workout entries
Error	#BA1A1A	Over-target state, delete
Surface	#F4FAF7	Screen background
On surface	#171D1B	Primary text

Typography: large displayMedium for the remaining-calories figure, titleLarge for section headers and labelLarge for chips and buttons. Calories are always shown with the "kcal" unit. Touch targets are at least 48 dp.

10.3 Figma versus implementation

> The Figma prototype is a design reference, and the built app extends it in places. Figma shows a single progress ring (consumed versus target) in a green palette. The implementation uses a dual-arc ring (blue outer arc for food, orange inner arc for exercise), the Outfit typeface, emoji icons on entry cards, and the weight and goal-plan dialog. The flow, screens, entry details and balance rules are the same. If asked, describe Figma as the guided-flow design and the app as the extended build.

11. Design Justification

Decision	Reasoning
Riverpod	Three screens must show the same balance. One shared provider removes callback passing and duplicated state, AsyncValue gives built-in loading and error handling, and providers can be tested without widgets.
AsyncNotifier + FutureProvider	Entries are loaded from and saved to storage asynchronously, so the calculation naturally follows an awaited result. This makes async/await necessary rather than decorative.
No backend	The brief asks for offline access. A server would add failure points, hosting and authentication with no benefit for a single-user tracker.
shared_preferences	The data per day is small. A JSON document per date is simple to read, write and test; a SQL database would be more than the problem needs.
MET formula	A standard, explainable method for estimating exercise burn from activity, weight and time, instead of a number typed in by the user.
Ring = consumed / target	Matches the wording of the problem statement. Burned calories are added back in the Remaining figure, so the balance is still net.
Material 3 theming	Consistent, accessible colour roles and components with little custom work; the status colour (green, red) tells the user their state without reading numbers.
Adaptive shell	One codebase supports phone, tablet and desktop; the rail layout appears only when there is space for it.
Undo on delete	Swipe gestures are easy to trigger by mistake; Undo makes the fast path safe.

12. Testing and Quality

- **test/calorie_calculator_test.dart** - zero-state defaults, net remaining balance, over-target boundary, deficit calculation and MET burn formulas.
- **test/widget_test.dart** - mobile layout rendering and tablet NavigationRail adaptation.

```
flutter analyze
flutter test
```

Manual acceptance checks

- Add a meal: the ring and totals change without a restart.
- Add a workout: burned and remaining update; the estimate matches the MET formula.
- Edit and delete an entry: balance updates immediately; Undo restores it.
- Close and reopen the app in airplane mode: data is still present.
- Resize the window or rotate the device: the layout switches between bottom bar and rail at 600 dp.

13. Build and Deployment

Target	Command	Output
Web	flutter build web --release --base-href "/"	build/web/
Android APK	flutter build apk --release	build/app/outputs/flutter-apk/app-release.apk
Android AAB	flutter build appbundle --release	build/app/outputs/bundle/release/app-release.aab
iOS	flutter build ipa --release	build/ios/archive/Runner.xcarchive
macOS / Windows / Linux	flutter build macos | windows | linux --release	build/ (per platform)

Web builds can be hosted on GitHub Pages, Vercel or Firebase Hosting (set build/web as the public directory). iOS builds need an Apple Developer account for signing and TestFlight upload.

14. Limitations and Future Scope

14.1 Known limitations

- **Fonts and offline use.** google_fonts downloads fonts at runtime by default. For guaranteed offline typography, bundle the Outfit font files as app assets.
- **"Deficit" wording.** The app calls a positive Remaining value a deficit. This is relative to the user's chosen target, not to true maintenance calories, so explain it that way if asked.
- **Estimates only.** MET-based burn and the 30 kcal per kg maintenance rule are approximations, not medical advice.
- **Local only.** Data is stored on one device; clearing app data removes it and there is no backup or sync.
- **Manual food entry.** Calories are typed or taken from 10 presets; there is no food database.

14.2 Future scope

- Weekly and monthly charts, streaks and trends.
- Food database or barcode scanning.
- Optional cloud backup and sync.
- Wearable and step-count integration.
- Reminders and notifications.

Appendix A. Project Files

Path	Role
lib/main.dart	App root, ProviderScope, MainNavigationShell
lib/screens/summary_screen.dart	Daily Summary screen
lib/screens/log_meal_screen.dart	Log Meal screen and sheet
lib/screens/log_workout_screen.dart	Log Workout screen and sheet
lib/providers/fitness_providers.dart	All Riverpod providers
lib/widgets/calorie_ring.dart	CalorieRing and _DualRingPainter
lib/widgets/entry_card.dart	EntryCard
lib/widgets/target_dialog.dart	TargetDialog
lib/widgets/quick_chips.dart	QuickMealChips
lib/widgets/date_header.dart	DateHeader
test/calorie_calculator_test.dart	Calculator unit tests
test/widget_test.dart	Responsive layout widget tests

LogRepository, CalorieCalculator and MetCalculator are referenced in the architecture; their file locations are not listed in the source notes for this document.